

C++ Set Practice Problems & Solutions

A collection of foundational C++ `std::set` and `std::unordered_set` practice problems with clean implementations.

1. Remove Duplicates and Print Sorted

Goal: Take a vector of unsorted integers with duplicates and display only the unique values in ascending order.

Key Concepts: `std::set` properties (automatic sorting and duplicate rejection).

```
#include <iostream>
#include <vector>
#include <set>
using namespace std;

int main() {
    vector<int> vNums = {8, 3, 5, 2, 8, 3, 1, 5, 9};

    // Inserting vector elements into set automatically
    // removes duplicates and sorts them
    set<int> sUniqueNums(vNums.begin(), vNums.end());

    cout << "Unique sorted elements: ";
    for (int num : sUniqueNums) {
        cout << num << " ";
    }
    cout << "\n";

    cout << "All done!" << endl;
}
```

Opportunities for Improvement

Write a template function to print a generic `set<T>` and a caption. For example `showSet(suniqueNums, "unique sorted elements")`

2. Check for Duplicate in a Vector

Goal: Determine whether any value appears at least twice in a given vector.

Key Concepts: `std::unordered_set` for $O(1)$ average lookup and insertion.

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool hasDuplicate(const vector<int>& vNums) {
    unordered_set<int> sSeen;

    for (int num : vNums) {
        if (sSeen.find(num) != sSeen.end()) {
            return true; // Duplicate found
        }
        sSeen.insert(num);
    }
    return false;
}

int main() {
    vector<int> vNums1 = { 1, 5, 8, 3, 2, 5 };
    vector<int> vNums2 = { 10, 20, 30, 40 };

    cout << "vNums1 has duplicate: " << (hasDuplicate(vNums1) ? "Yes" : "No") << "\n";
    cout << "vNums2 has duplicate: " << (hasDuplicate(vNums2) ? "Yes" : "No") << "\n";

    cout << "All done!" << endl;
}
```

Alternative Approach

Initialize the set `sSeen` with the vector data. Compare whether the set and the vector have the same size. If the size matches nothing was deleted from the vector, so it has no duplicates.

3. Intersection of Two Arrays

Goal: Given two integer vectors, find all common elements that appear in both vectors. Each element in the result must be unique.

Key Concepts: `std::unordered_set` lookup, set filtering.

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

vector<int> intersection(const vector<int>& vNums1, const vector<int>& vNums2) {
    unordered_set<int> set1(vNums1.begin(), vNums1.end());
    unordered_set<int> resultSet;

    for (int num : vNums2) {
        if (set1.count(num)) {
            resultSet.insert(num);
        }
    }

    return vector<int>(resultSet.begin(), resultSet.end());
}

int main() {
    vector<int> vNums1 = {4, 9, 5, 4};
    vector<int> vNums2 = {9, 4, 9, 8, 4};

    vector<int> result = intersection(vNums1, vNums2);

    cout << "Intersection: ";
    for (int num : result) {
        cout << num << " ";
    }
    cout << "\n";

    cout << "All done!" << endl;
}
```

4. Two Sum (Target Pair Detection)

Goal: Check if there exist two numbers in a vector that add up to a given target sum.

Key Concepts: Complement search, `std::unordered_set` hash lookup in $O(n)$ total time.

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

bool hasTwoSum(const vector<int>& vNums, int target) {
    unordered_set<int> sSeen;

    for (int num : vNums) {
        int complement = target - num;
        if (sSeen.count(complement) == 1) {
            cout << "Found pair: " << num << " + "
                << complement
                << " = " << target << "\n";
            return true;
        }
        sSeen.insert(num);
    }
    return false;
}

int main() {
    vector<int> vNums = { 2, 7, 11, 15 };

    int target = 9;

    if (!hasTwoSum(vNums, target)) {
        cout << "No pair adds up to " << target << "\n";
    }

    cout << "All done!" << endl;
}
```

5. Find All Disappeared Numbers in a Range

Goal: Given a vector containing numbers from range [1, N], find all integers in that range that do not appear in the vector.

Key Concepts: Range lookup, `std::unordered_set` membership check.

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

vector<int> findMissingNumbers(const vector<int>& vNums, int n) {

    unordered_set<int> sPresentVectorValues(vNums.begin(), vNums.end());
    vector<int> vMissing;

    for (int i = 1; i <= n; ++i) {
        if (sPresentVectorValues.find(i) == sPresentVectorValues.end()) {
            vMissing.push_back(i);
        }
    }

    return vMissing;
}

int main() {
    //vNums is not sorted and may have duplicates
    vector<int> vNums = { 4, 3, 2, 7, 8, 2, 3, 1 };

    int n = 8; // Expected range 1 to 8

    vector<int> vMissing = findMissingNumbers(vNums, n);

    cout << "Missing numbers in range 1 to " << n << ": ";
    for (int num : vMissing) {
        cout << num << ", ";
    }
    cout << "\n";

    cout << "All done!" << endl;
}
```